

DB Navigator Demo

REST API Backend Projekt

Technische Projektdokumentation

Technologie	Spring Boot 3.5 + Java 17
Datenbank	PostgreSQL 15 (Docker)
API Dokumentation	OpenAPI 3.0 / Swagger UI
Build Tool	Maven
Datum	11.05.2026

Inhaltsverzeichnis

1. Projektübersicht
2. Technologie-Stack
3. Projektstruktur
4. Entwicklungsumgebung
5. Datenmodell
6. REST API Endpunkte
7. Code-Beispiele
8. Docker Konfiguration
9. OpenAPI / Swagger
10. Tests
11. Interview-Relevante Themen

1. Projektübersicht

Das DB Navigator Demo Projekt ist eine REST API Backend-Anwendung, die als Vorbereitung für eine Backend-Entwickler Position bei DB Fernverkehr AG entwickelt wurde. Das Projekt demonstriert moderne Java-Entwicklung mit Spring Boot und implementiert typische Funktionen einer Bahnhofssuche und Zugverbindungsabfrage.

Kernfunktionen:

- CRUD-Operationen für Bahnhöfe (Stations)
- CRUD-Operationen für Zugverbindungen (TrainConnections)
- Verbindungssuche zwischen Bahnhöfen
- Abfahrtstafel für einzelne Bahnhöfe
- Suche nach Zugnummer
- Filterung nach Zugtyp (ICE, IC, RE, RB, S)
- Validierung mit Bean Validation
- Exception Handling mit standardisierten Fehlerantworten
- OpenAPI/Swagger Dokumentation

2. Technologie-Stack

Komponente	Technologie	Version
Backend Framework	Spring Boot	3.5.10
Programmiersprache	Java	17 LTS
Build Tool	Maven	3.9.x
Datenbank	PostgreSQL	15
ORM	Hibernate / JPA	6.x
API Dokumentation	springdoc-openapi	2.7.0
Containerisierung	Docker	28.x
Code Generation	Lombok	via Spring Boot
Validierung	Jakarta Bean Validation	3.0

Relevanz für DB Fernverkehr:

Der gewählte Stack entspricht den Anforderungen der Stellenbeschreibung: Java 17, Spring Framework (Boot), REST APIs, JPA/Hibernate, PostgreSQL, Docker. Diese Technologien werden bei DB Navigator und bahn.de produktiv eingesetzt.

3. Projektstruktur

```
db-navigator-demo/
├── src/main/java/com/db/navigator/
│   ├── DbNavigatorDemoApplication.java
│   ├── config/
│   │   └── OpenApiConfig.java
│   ├── controller/
│   │   ├── StationController.java
│   │   └── TrainConnectionController.java
│   ├── service/
│   │   ├── StationService.java
│   │   └── TrainConnectionService.java
│   ├── repository/
│   │   ├── StationRepository.java
│   │   └── TrainConnectionRepository.java
│   ├── model/
│   │   ├── entity/
│   │   │   ├── Station.java
│   │   │   ├── TrainConnection.java
│   │   │   └── TrainType.java
│   │   └── dto/
│   │       ├── StationDTO.java
│   │       └── TrainConnectionDTO.java
│   ├── exception/
│   │   ├── ResourceNotFoundException.java
│   │   ├── DuplicateResourceException.java
│   │   └── GlobalExceptionHandler.java
│   └── src/main/resources/
│       └── application.properties
├── src/test/java/
│   └── com/db/navigator/service/
│       └── StationServiceTest.java
├── docker-compose.yml
└── pom.xml
```

Architektur-Pattern:

Das Projekt folgt dem klassischen Schichtenmodell (Layered Architecture) mit klarer Trennung von Controller, Service und Repository. DTOs werden verwendet um die Domänenobjekte von der API-Schicht zu entkoppeln.

4. Entwicklungsumgebung

Voraussetzungen:

- Java 17 LTS (OpenJDK oder Oracle JDK)
- Maven 3.9+
- Docker Desktop
- Git
- IDE: IntelliJ IDEA (empfohlen) oder VS Code

Projekt starten:

```
# 1. Repository klonen
git clone <repository-url>
cd db-navigator-demo

# 2. PostgreSQL Container starten
docker-compose up -d

# 3. Anwendung starten
./mvnw spring-boot:run

# 4. Swagger UI öffnen
# http://localhost:8080/swagger-ui/index.html
```

5. Datenmodell

5.1 Station Entity

```
@Entity
@Table(name = "stations")
@Data @Builder @NoArgsConstructor @AllArgsConstructor
public class Station {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 10)
    private String code; // z.B. "FRA", "BER"

    @Column(nullable = false)
    private String name;

    @Column(nullable = false)
    private String city;

    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
}
```

5.2 TrainConnection Entity

```
@Entity
@Table(name = "train_connections")
@Data @Builder @NoArgsConstructor @AllArgsConstructor
public class TrainConnection {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String trainNumber; // z.B. "ICE 123"

    @Enumerated(EnumType.STRING)
    private TrainType trainType; // ICE, IC, EC, RE, RB, S

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "departure_station_id")
    private Station departureStation;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "arrival_station_id")
    private Station arrivalStation;

    private LocalDateTime departureTime;
    private LocalDateTime arrivalTime;
    private BigDecimal price;
    private Integer availableSeats;
    private String platform;
}
```

5.3 TrainType Enum

```
public enum TrainType {
    ICE, // Intercity-Express
    IC,  // Intercity
    EC,  // Eurocity
    RE,  // Regional-Express
    RB,  // Regionalbahn
    S    // S-Bahn
}
```

6. REST API Endpunkte

6.1 Station API

Methode	Endpunkt	Beschreibung
GET	/api/v1/stations	Alle Bahnhöfe abrufen
GET	/api/v1/stations/{id}	Bahnhof nach ID
GET	/api/v1/stations/code/{code}	Bahnhof nach Code
GET	/api/v1/stations/search?name=	Bahnhöfe suchen
POST	/api/v1/stations	Neuen Bahnhof anlegen
PUT	/api/v1/stations/{id}	Bahnhof aktualisieren
DELETE	/api/v1/stations/{id}	Bahnhof löschen

6.2 TrainConnection API

Methode	Endpunkt	Beschreibung
GET	/api/v1/connections	Alle Verbindungen
GET	/api/v1/connections/{id}	Verbindung nach ID
GET	/api/v1/connections/search	Verbindungen suchen
GET	/api/v1/connections/departures/{code}	Abfahrtstafel
GET	/api/v1/connections/train/{number}	Nach Zugnummer suchen
POST	/api/v1/connections	Verbindung erstellen
PUT	/api/v1/connections/{id}	Verbindung aktualisieren
DELETE	/api/v1/connections/{id}	Verbindung löschen

Suchparameter für /connections/search:

- from (required): Abfahrtsbahnhof Code (z.B. FRA)
- to (required): Ankunftsbahnhof Code (z.B. BER)
- departure (required): Früheste Abfahrtszeit (ISO 8601)
- type (optional): Zugtyp Filter (ICE, IC, RE, etc.)

7. Code-Beispiele

7.1 Repository mit Custom Queries

```
@Repository
public interface TrainConnectionRepository
    extends JpaRepository<TrainConnection, Long> {

    @Query("SELECT c FROM TrainConnection c " +
        "JOIN FETCH c.departureStation " +
        "JOIN FETCH c.arrivalStation " +
        "WHERE c.departureStation.code = :fromCode " +
        "AND c.arrivalStation.code = :toCode " +
        "AND c.departureTime >= :minDeparture " +
        "ORDER BY c.departureTime")
    List<TrainConnection> findConnections(
        @Param("fromCode") String fromCode,
        @Param("toCode") String toCode,
        @Param("minDeparture") LocalDateTime minDeparture);
}
```

7.2 Service Layer mit Transaktionen

```
@Service
@RequiredArgsConstructor
@Transactional(readOnly = true)
public class StationService {
    private final StationRepository stationRepository;

    @Transactional
    public StationDTO create(StationDTO dto) {
        if (stationRepository.existsByCode(dto.getCode())) {
            throw new DuplicateResourceException(
                "Station mit Code " + dto.getCode() +
                " existiert bereits");
        }
        Station station = toEntity(dto);
        Station saved = stationRepository.save(station);
        return toDTO(saved);
    }
}
```

7.3 Controller mit OpenAPI Annotationen

```
@RestController
@RequestMapping("/api/v1/stations")
@RequiredArgsConstructor
@Tag(name = "Bahnhöfe",
    description = "API zur Verwaltung von Bahnhöfen")
public class StationController {

    @Operation(summary = "Bahnhof nach ID suchen")
    @ApiResponse({
        @ApiResponse(responseCode = "200",
            description = "Bahnhof gefunden"),
        @ApiResponse(responseCode = "404",
            description = "Bahnhof nicht gefunden")
    })
    @GetMapping("/{id}")
    public ResponseEntity<StationDTO> getStationById(
        @Parameter(description = "ID des Bahnhofs")
        @PathVariable Long id) {
        return ResponseEntity.ok(stationService.findById(id));
    }
}
```

8. Docker Konfiguration

docker-compose.yml:

```
version: '3.8'
services:
  postgres:
    image: postgres:15-alpine
    container_name: db-navigator-postgres
    environment:
      POSTGRES_DB: dbnavigator
      POSTGRES_USER: dbuser
      POSTGRES_PASSWORD: dbpass
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U dbuser -d dbnavigator"]
      interval: 10s
      timeout: 5s
      retries: 5

volumes:
  postgres_data:
```

application.properties:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/dbnavigator
spring.datasource.username=dbuser
spring.datasource.password=dbpass
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
server.port=8080
management.endpoints.web.exposure.include=health,info
```

9. OpenAPI / Swagger

Konfiguration:

```
@Configuration
public class OpenApiConfig {
    @Bean
    public OpenAPI dbNavigatorOpenAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("DB Navigator API")
                .description("REST API fuer Bahnhofssuche " +
                    "und Zugverbindungen")
                .version("1.0.0")
                .contact(new Contact()
                    .name("DB Fernverkehr AG")
                    .email("api@bahn.de")))
            .servers(List.of(
                new Server()
                    .url("http://localhost:8080")
                    .description("Entwicklung")));
    }
}
```

Zugriff:

- Swagger UI: <http://localhost:8080/swagger-ui/index.html>
- OpenAPI JSON: <http://localhost:8080/v3/api-docs>
- OpenAPI YAML: <http://localhost:8080/v3/api-docs.yaml>

10. Tests

10.1 Unit Test Beispiel

```
@ExtendWith(MockitoExtension.class)
@DisplayName("StationService Tests")
class StationServiceTest {

    @Mock
    private StationRepository stationRepository;

    @InjectMocks
    private StationService stationService;

    @Test
    @DisplayName("sollte Bahnhof nach ID finden")
    void shouldFindStationById() {
        // Given
        Station station = Station.builder()
            .id(1L).code("FRA")
            .name("Frankfurt Hbf").build();
        when(stationRepository.findById(1L))
            .thenReturn(Optional.of(station));

        // When
        StationDTO result = stationService.findById(1L);

        // Then
        assertThat(result.getCode()).isEqualTo("FRA");
    }

    @Test
    @DisplayName("sollte Exception werfen bei doppeltem Code")
    void shouldThrowExceptionWhenCodeExists() {
        // Given
        StationDTO dto = StationDTO.builder()
            .code("FRA").build();
        when(stationRepository.existsByCode("FRA"))
            .thenReturn(true);

        // When & Then
        assertThatThrownBy(() -> stationService.create(dto))
            .isInstanceOf(DuplicateResourceException.class);
    }
}
```

Tests ausführen:

```
# Alle Tests ausführen
./mvnw test

# Einzelne Testklasse
./mvnw test -Dtest=StationServiceTest

# Mit Coverage Report
./mvnw test jacoco:report
```

11. Interview-Relevante Themen

Dieses Projekt demonstriert folgende Kenntnisse:

Thema	Demonstration im Projekt
Java 17	Records, var, Stream API, Optional
Spring Boot 3.x	Auto-Configuration, Starter Dependencies
REST API Design	CRUD, Query Parameters, Path Variables
JPA/Hibernate	Entities, Relationships, JPQL Queries
Validierung	Bean Validation, Custom Messages
Exception Handling	@RestControllerAdvice, Custom Exceptions
Transaktionen	@Transactional, readOnly Optimization
Testing	JUnit 5, Mockito, AssertJ
Docker	Compose, Volumes, Health Checks
API Dokumentation	OpenAPI 3.0, Swagger UI
Lombok	@Data, @Builder, @RequiredArgsConstructor

Typische Interview-Fragen:

1. Erklären Sie den Unterschied zwischen @Controller und @RestController
2. Wie funktioniert Dependency Injection in Spring?
3. Was ist der Unterschied zwischen FetchType.LAZY und EAGER?
4. Wie würden Sie diese API skalieren?
5. Welche Sicherheitsmaßnahmen würden Sie implementieren?
6. Erklären Sie das N+1 Problem und wie Sie es lösen
7. Wie funktioniert @Transactional unter der Haube?